```metal
//
//  File.metal
//  ArduiMatorNew
//
//  Created by AnshuDiya on 9/14/26.
//

#include <metal_stdlib>
using namespace metal;

//converts a given point from viewSpace into NDC
inline float viewSpaceToNDC(bool xCoordsOrYcoords, float
viewSpaceCoords, int SwiftFrameSize){
    if(xCoordsOrYcoords){
        return ((viewSpaceCoords/SwiftFrameSize)-1.0);
    } else{
        return (1.0-(viewSpaceCoords/SwiftFrameSize));
    }
}

struct NormalVectorMetadata{//refer to pages for documentation
    float2 currentPixelCoords;
    float slopeBetweenOrigPoints;
    float invSlope;
    float b2;
    float b1;
    float xCoordOfIntersection;
        float yCoordOfIntersection;
    float distFromLineSeg;
    float pixelLookupIndiceVal;
    float dzdx;
    float dzdy;
    float intersectionPointCoords;
        float3 T1;
        float3 T2;
            float3 NormalVector;
};

struct FragInfoStruct{
    float4 position [[position]];
    float2 SwiftFrameSize;
    float2 point1 [[flat]];
    float2 point2 [[flat]];
    float reflectivity [[flat]];//hardcoded for now
    float3 surfaceClr [[flat]];//hardcoded for now
};
```

```
struct LightSourceInfo{
    float lightIntensity;
    float3 lightPosition;
};

struct DerVertexOutput{
    float4 position [[position]];
    float pointSize [[point_size]];
};

vertex FragInfoStruct basic_vertex(
const device packed_float2* vertexArray[[buffer(0)]], const
device packed_float4* clrAndReflectivity [[buffer(3)]],//
contains clr and reflecivity values for that specific point
unsigned int vertexGid [[vertex_id]]) {
    FragInfoStruct FragInf;
    FragInf.position = float4(vertexArray[vertexGid],1.0,1.0);
    FragInf.point1 = vertexArray[vertexGid];
    FragInf.point2 = vertexArray[vertexGid+1];//beginning and
ending points of line segment
    if((FragInf.point2.x > 1.0)||(FragInf.point1.x > 1.0))
{FragInf.point2 = 10.0; FragInf.point1 = 10.0;}//if the line
segment's point 2 or point 1 reached is the end of the screen
clip both
    FragInf.SwiftFrameSize = float2(750.0f,750.0f);//hardcoded
here
    FragInf.reflectivity = clrAndReflectivity[vertexGid].x;
    FragInf.surfaceClr =
float3(clrAndReflectivity[vertexGid].y,clrAndReflectivity[vertex
Gid].z,clrAndReflectivity[vertexGid].w);//clr values
    return FragInf;
}

fragment float4 basicFragment(
FragInfoStruct InfoStruct [[stage_in]], const device
packed_float4* lightInfoBuffer [[buffer(1)]], const device
float* HeightLookupTable [[buffer(2)]]){

    unsigned int lightCount = (int)lightInfoBuffer[0].x;//the
first index of lightInfoBuffer contains the number of lights.
Could also be zero!
        float HeightTableCount = (int)HeightLookupTable[0];//
first index contains number of height values.
        float maxDist = sqrt(8.0f);//the maximum distance that
is permissible between a line segment and a point

    NormalVectorMetadata Nvec;
```

```
    float currentX = viewSpaceToNDC(true, InfoStruct.position.x,
InfoStruct.SwiftFrameSize.x);
    float currentY = viewSpaceToNDC(false,
InfoStruct.position.y, InfoStruct.SwiftFrameSize.y);
    Nvec.currentPixelCoords = float2(currentX,currentY);

    Nvec.slopeBetweenOrigPoints = ((InfoStruct.point2.y-
InfoStruct.point1.y)/(InfoStruct.point2.x-InfoStruct.point1.x));
        Nvec.invSlope = -1.0f/Nvec.slopeBetweenOrigPoints;
    Nvec.b2 = Nvec.currentPixelCoords.y-
(Nvec.invSlope*Nvec.currentPixelCoords.x);//intercept of the
perpendicular line
    Nvec.b1 = InfoStruct.point1.y -
(Nvec.slopeBetweenOrigPoints*InfoStruct.point1.x);//intercept of
line going thorugh line segment

    Nvec.xCoordOfIntersection = ((Nvec.b2-Nvec.b1)/
(Nvec.invSlope+Nvec.slopeBetweenOrigPoints));
        Nvec.yCoordOfIntersection =
(Nvec.slopeBetweenOrigPoints*Nvec.xCoordOfIntersection)+Nvec.b1;

    Nvec.distFromLineSeg =
distance(float2(Nvec.xCoordOfIntersection,Nvec.yCoordOfIntersect
ion),Nvec.currentPixelCoords);//distance from point of
intersection and current pixel
    int indiceToLookup =
(int(clamp(Nvec.distFromLineSeg,0.0f,HeightTableCount)));
    Nvec.pixelLookupIndiceVal =
HeightLookupTable[indiceToLookup];
    Nvec.pixelLookupIndiceVal += ((Nvec.pixelLookupIndiceVal-
(int)Nvec.pixelLookupIndiceVal))*((HeightLookupTable[indiceToLoo
kup+1]-HeightLookupTable[indiceToLookup])/(maxDist/
HeightTableCount));

    Nvec.dzdx = dfdx(Nvec.pixelLookupIndiceVal);
    Nvec.dzdy = dfdy(Nvec.pixelLookupIndiceVal);
        Nvec.T1 = float3(1,0,Nvec.dzdx);
        Nvec.T2 = float3(0,1,Nvec.dzdy);
    Nvec.NormalVector = cross(Nvec.T1, Nvec.T2);

    float totalBrightness = 0.0f;
    for(unsigned int k = 0; k < lightCount; k++){
        //when performing lighting calculations, make a new
struct with every light source
        LightSourceInfo LSinfo;
        LSinfo.lightPosition = float3((lightInfoBuffer[k].x -
Nvec.currentPixelCoords.x),(lightInfoBuffer[k].y -
```

```
Nvec.currentPixelCoords.y),(lightInfoBuffer[k].z -
Nvec.pixelLookupIndiceVal));
        LSinfo.lightPosition = normalize(LSinfo.lightPosition);
        float brightness = max(dot(Nvec.NormalVector,
LSinfo.lightPosition),0.0);
        LSinfo.lightIntensity = lightInfoBuffer[k].w;//light
intensity stored in the 4th element
        totalBrightness += brightness*LSinfo.lightIntensity;
    }
    float3 finalPxlClr =
InfoStruct.reflectivity*InfoStruct.surfaceClr*totalBrightness;
    return float4(finalPxlClr,1.0);

    //return float4(0.3,0.6,0.4,1.0);
}
```